Heap / Priority Queue — Complete Guide

1. What is a Heap?

A Heap is a special tree-based data structure that follows two rules:

1. It must be a Complete Binary Tree 2. It must satisfy the Heap Property

There are two main types:

- Min Heap → smallest element is at the root
- Max Heap → largest element is at the root

Example of a Min Heap:

```
        10
       /  \
     20    30
    / \    /
   40 50  60
```

The important property is:

```
Parent <= Children
```

For a Max Heap:

```
        60
       /  \
     50    40
    / \    /
   20 30  10
```

Property:

```
Parent >= Children
```

2. Why Do We Need a Heap?

Suppose we have:

```
10 50 20 40 30
```

And repeatedly need the smallest element.

A normal array:

```
10 50 20 40 30
```

Finding minimum:

```
O(n)
```

If we sort:

```
10 20 30 40 50
```

Finding minimum:

```
O(1)
```

But inserting a new element may require shifting/rearranging.

A heap gives us:

```
Get minimum       O(1)
Insert            O(log n)
Delete minimum    O(log n)
```

That's why heaps are extremely useful when we repeatedly need the minimum or maximum.

3. Heap Is a Complete Binary Tree

This concept is very important.

A Complete Binary Tree means:

- Every level is completely filled except possibly the last.
- The last level is filled from left to right.

Valid:

```
        10
      /  \
    20    30
   / \   /
  40 50 60
```

Invalid:

```
        10
      /  \
    20    30
      \
       40
```

Why?

Because the last level isn't filled from left to right.

4. Heap Is NOT a Binary Search Tree

This is a common confusion.

BST

For every node:

```
Left < Root < Right
```

Example:

```
        40
       /  \
     20    60
    / \    / \
   10 30  50 70
```

Min Heap

Only:

```
Parent <= Children
```

Example:

```
        10
       /  \
     30    20
    / \    /
   50 40  60
```

Notice:

```
30 > 20
```

That's perfectly valid in a heap.

So:

Heap does NOT maintain sorted order.

5. Types of Heap

There are two important types.

Min Heap

Smallest element at root.

```
        10
       /  \
     20    30
    / \    /
   40 50  60
```

Root:

```
10
```

Max Heap

Largest element at root.

```
        60
       /  \
     50    40
    / \    /
   20 30  10
```

Root:

```
60
```

6. Heap Representation Using Array

This is one of the most important concepts.

Although a heap is conceptually a tree, we normally store it in an array.

Example:

```
        10
       /  \
     20    30
    / \    /
   40 50  60
```

Array:

```
[10, 20, 30, 40, 50, 60]
```

No pointers are required.

7. Why Can We Store a Heap in an Array?

Because the tree is complete.

For an array using 0-based indexing:

```
Parent = (i - 1) / 2

Left Child = 2 * i + 1

Right Child = 2 * i + 2
```

For example:

```
[10, 20, 30, 40, 50, 60]
```

Indexes:

```
index
 0   1   2   3   4   5

[10, 20, 30, 40, 50, 60]
```

For index 0:

```
left = 2(0)+1
     = 1

right = 2(0)+2
      = 2
```

So:

```
10
├── 20
└── 30
```

For index 1:

```
left = 3
right = 4
```

Therefore:

```
20
├── 40
└── 50
```

8. Important Heap Formulas

For 0-based indexing:

Operation	Formula
Parent	(i - 1) / 2
Left child	2 * i + 1
Right child	2 * i + 2

Remember these three.

9. Height of Heap

A heap is a complete binary tree.

Height:

```
O(log n)
```

Why?

Because each level approximately doubles the number of nodes.

```
Level 0 → 1
Level 1 → 2
Level 2 → 4
Level 3 → 8
```

Therefore:

```
n ≈ 2^h
```

Taking log:

```
h ≈ log₂ n
```

This is why insertion and deletion take:

```
O(log n)
```

10. Heapify

Heapify means restoring the heap property.

There are two important operations:

Min Heap

```
minHeapify()
```

Max Heap

```
maxHeapify()
```

Heapify is used when a node violates the heap property.

11. Min Heapify

Suppose:

```
        50
      /   \
    20     30
```

This violates Min Heap because:

```
50 > 20
50 > 30
```

We compare:

```
50
20
30
```

Smallest is:

```
20
```

Swap:

```
        20
       /  \
     50    30
```

Now heap property is restored for this portion.

12. Java Min Heap Implementation

Let's implement a Min Heap from scratch.

```java
class MinHeap {

    int[] heap;
    int size;

    MinHeap(int capacity) {
        heap = new int[capacity];
        size = 0;
    }

    int parent(int i) {
        return (i - 1) / 2;
    }

    int leftChild(int i) {
        return 2 * i + 1;
    }

    int rightChild(int i) {
        return 2 * i + 2;
    }

    void swap(int i, int j) {
        int temp = heap[i];
        heap[i] = heap[j];
        heap[j] = temp;
    }
}
```

13. Insert in Min Heap

Suppose we have:

```
        10
       /  \
     20    30
```

Insert:

```
5
```

First place it at the next available position:

```
        10
       /  \
     20    30
    /
   5
```

But:

```
5 < 20
```

So heap property is violated.

We move 5 upward.

This operation is called:

Heapify Up / Bubble Up / Sift Up

14. Heapify Up

Before:

```
        10
       /  \
     20    30
    /
   5
```

Swap:

```
        10
       /  \
      5    30
     /
    20
```

Again:

```
5 < 10
```

Swap:

```
        5
       / \
     10   30
    /
   20
```

Done.

15. Java Min Heap Insert

```
void insert(int value) {

    heap[size] = value;

    int current = size;

    size++;

    while (current > 0) {

        int parentIndex = parent(current);

        if (heap[parentIndex] <= heap[current]) {
            break;
        }

        swap(parentIndex, current);

        current = parentIndex;
    }
}
```

The important part:

```
while (current > 0)
```

We keep moving toward the root.

16. Time Complexity of Insert

At worst, the element travels from:

```
leaf → root
```

Height of heap:

```
O(log n)
```

Therefore:

```
Insert = O(log n)
```

17. Get Minimum

In a Min Heap:

```
root = minimum
```

So:

```
int peek() {
    return heap[0];
}
```

Time:

```
O(1)
```

Because we don't search.

18. Delete Minimum

This is one of the most important heap operations.

Suppose:

```
        10
       /  \
     20    30
    / \    /
   40 50  60
```

Array:

```
[10, 20, 30, 40, 50, 60]
```

We want to remove:

```
10
```

We can't simply leave the root empty.

So:

Step 1

Take the last element:

```
60
```

Step 2

Put it at root:

```
        60
       /  \
     20    30
    / \
   40 50
```

Step 3

Now heap property is broken.

```
60 > 20
60 > 30
```

So we perform:

Heapify Down / Sift Down / Bubble Down

19. Heapify Down

Compare the node with its children.

```
       60
      /  \
    20    30
```

Smallest child:

```
20
```

Swap:

```
       20
      /  \
    60    30
```

Now:

```
60
```

has children:

```
40
50
```

Smallest is 40.

Swap:

```
       20
      /  \
    40    30
   / \
  60 50
```

Done.

20. Java Delete Min

```java
int deleteMin() {

    if (size == 0) {
        throw new RuntimeException("Heap is empty");
    }

    int min = heap[0];

    heap[0] = heap[size - 1];

    size--;

    int current = 0;

    while (true) {

        int left = leftChild(current);
        int right = rightChild(current);

        int smallest = current;

        if (left < size && heap[left] < heap[smallest]) {
            smallest = left;
        }

        if (right < size && heap[right] < heap[smallest]) {
            smallest = right;
        }

        if (smallest == current) {
            break;
        }

        swap(current, smallest);

        current = smallest;
    }

    return min;
}
```

21. Why Do We Compare Both Children?

Suppose:

```
       50
      /  \
    20    10
```

If we only check the left:

```
50 > 20
```

we might swap with 20.

But the correct child is:

```
10
```

because 10 is smaller.

Therefore:

```
if (left < size && heap[left] < heap[smallest])
```

and

```
if (right < size && heap[right] < heap[smallest])
```

Then choose the smaller child.

22. Complexity of Delete Min

Again, the element can travel:

```
root → leaf
```

Height:

```
O(log n)
```

Therefore:

```
Delete Min = O(log n)
```

23. Complete Min Heap Implementation

```
class MinHeap {

    int[] heap;
    int size;

    MinHeap(int capacity) {
        heap = new int[capacity];
        size = 0;
    }

    int parent(int i) {
        return (i - 1) / 2;
    }

    int leftChild(int i) {
        return 2 * i + 1;
    }

    int rightChild(int i) {
        return 2 * i + 2;
    }

    void swap(int i, int j) {
        int temp = heap[i];
        heap[i] = heap[j];
        heap[j] = temp;
    }

    void insert(int value) {

        heap[size] = value;

        int current = size;

        size++;

        while (current > 0) {

            int parentIndex = parent(current);

            if (heap[parentIndex] <= heap[current]) {
                break;
            }

            swap(parentIndex, current);

            current = parentIndex;
        }
    }

    int peek() {

        if (size == 0) {
            throw new RuntimeException("Heap is empty");
        }

        return heap[0];
    }

    int deleteMin() {
```

```
        if (size == 0) {
            throw new RuntimeException("Heap is empty");
        }

        int min = heap[0];

        heap[0] = heap[size - 1];

        size--;

        int current = 0;

        while (true) {

            int left = leftChild(current);
            int right = rightChild(current);

            int smallest = current;

            if (left < size && heap[left] < heap[smallest]) {
                smallest = left;
            }

            if (right < size && heap[right] < heap[smallest]) {
                smallest = right;
            }

            if (smallest == current) {
                break;
            }

            swap(current, smallest);

            current = smallest;
        }

        return min;
    }
}
```

24. Max Heap

Everything is almost identical.

The difference:

Min Heap:

```
Parent <= Children
```

Max Heap:

```
Parent >= Children
```

Example:

```
        50
       /  \
     30    40
    / \
  10 20
```

25. Max Heap Insert

Suppose:

```
        50
       /  \
     30    40
```

Insert:

```
60
```

First:

```
        50
       /  \
     30    40
    /
  60
```

60 > 30, swap:

```
        50
       /  \
     60    40
    /
  30
```

Then:

```
60 > 50
```

Swap:

```
        60
       /  \
     50    40
    /
  30
```

26. Java Max Heap Insert

```
void insert(int value) {

    heap[size] = value;

    int current = size;

    size++;

    while (current > 0) {

        int parentIndex = parent(current);

        if (heap[parentIndex] >= heap[current]) {
            break;
        }

        swap(parentIndex, current);

        current = parentIndex;
    }
}
```

Notice the difference:

Min Heap:

```
heap[parentIndex] <= heap[current]
```

Max Heap:

```
heap[parentIndex] >= heap[current]
```

27. Max Heap Delete

For Max Heap, we remove:

```
maximum
```

which is at:

```
heap[0]
```

Then move the last element to the root and perform heapify down.

The difference is that we choose the larger child.

```
int deleteMax() {

    if (size == 0) {
        throw new RuntimeException("Heap is empty");
    }

    int max = heap[0];

    heap[0] = heap[size - 1];

    size--;
```

```
    int current = 0;

    while (true) {

        int left = leftChild(current);
        int right = rightChild(current);

        int largest = current;

        if (left < size && heap[left] > heap[largest]) {
            largest = left;
        }

        if (right < size && heap[right] > heap[largest]) {
            largest = right;
        }

        if (largest == current) {
            break;
        }

        swap(current, largest);

        current = largest;
    }

    return max;
}
```

28. Heap Operations Complexity

Operation	Min Heap	Max Heap
Peek	O(1)	O(1)
Insert	O(log n)	O(log n)
Delete root	O(log n)	O(log n)
Search	O(n)	O(n)

Important:

Heap is not designed for fast arbitrary searching.

29. Building a Heap

Suppose we have:

```
[50, 20, 40, 10, 30]
```

This isn't necessarily a heap.

We can convert it into a heap.

There are two approaches.

Approach 1

Insert elements one by one.

Each insertion:

```
O(log n)
```

Total:

```
O(n log n)
```

Approach 2

Use Bottom-Up Heap Construction.

Total:

```
O(n)
```

This is very important.

30. Build Heap Using Heapify

For a Max Heap:

```
[10, 30, 20, 5, 15, 40]
```

The last non-leaf node is:

```
(n / 2) - 1
```

For n = 6:

```
6 / 2 - 1
= 2
```

So start from index 2.

Then:

```
2
1
0
```

Heapify each node downward.

31. Java Build Max Heap

```
void buildMaxHeap(int[] arr) {

    int n = arr.length;

    for (int i = n / 2 - 1; i >= 0; i--) {
        maxHeapify(arr, n, i);
    }
}
```

Heapify:

```
void maxHeapify(int[] arr, int n, int i) {

    int largest = i;

    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest]) {
        largest = left;
    }

    if (right < n && arr[right] > arr[largest]) {
        largest = right;
    }

    if (largest != i) {

        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        maxHeapify(arr, n, largest);
    }
}
```

32. Why Start From n/2 - 1?

All nodes from:

```
n / 2
```

to:

```
n - 1
```

are leaf nodes.

Leaves already satisfy heap property because they have no children.

Therefore we only need to heapify:

```
n/2 - 1 → 0
```

33. Priority Queue

Now we come to the most important practical application of heaps.

A Priority Queue is a data structure where elements are removed according to their priority, rather than simply according to insertion order.

Normal Queue:

```
First In → First Out
```

Priority Queue:

```
Highest/lowest priority → removed first
```

34. Normal Queue vs Priority Queue

Normal Queue:

```
10 → 20 → 30 → 40
```

Remove:

```
10
```

because it arrived first.

Priority Queue:

```
10 priority = 5
20 priority = 2
30 priority = 1
```

If smaller number means higher priority:

```
30
```

is removed first.

35. Java PriorityQueue

Java already provides:

```
PriorityQueue
```

from:

```
java.util.PriorityQueue
```

By default:

Java PriorityQueue is a Min Heap.

Example:

```java
import java.util.PriorityQueue;

PriorityQueue<Integer> pq = new PriorityQueue<>();

pq.add(50);
pq.add(20);
pq.add(40);
pq.add(10);
pq.add(30);
```

Internally it maintains heap order.

36. peek()

Returns the highest-priority element without removing it.

For default PriorityQueue:

```java
System.out.println(pq.peek());
```

Output:

```
10
```

Time:

```
O(1)
```

37. poll()

Removes and returns the highest-priority element.

```java
System.out.println(pq.poll());
```

Output:

```
10
```

Now:

```
20
30
40
50
```

Time:

```
O(log n)
```

38. add() / offer()

Both insert an element.

```
pq.add(25);
```

or:

```
pq.offer(25);
```

Time:

```
O(log n)
```

39. remove()

Removes the head.

```
pq.remove();
```

For an empty queue, remove() throws an exception.

40. poll() vs remove()

Important difference.

If empty:

```
pq.remove();
```

throws an exception.

But:

```
pq.poll();
```

returns:

```
null
```

So:

Method	Empty
remove()	Exception
poll()	null

41. element() vs peek()

Both return the head without removing.

But when empty:

```
element()
```

throws exception.

Whereas:

```
peek()
```

returns:

```
null
```

42. PriorityQueue Important Methods

Method	Purpose	Complexity
add()	Insert	O(log n)
offer()	Insert	O(log n)
peek()	View top	O(1)
poll()	Remove top	O(log n)
remove()	Remove top	O(log n)
element()	View top	O(1)
size()	Number of elements	O(1)
isEmpty()	Check empty	O(1)
clear()	Remove all	O(n)

43. Max Heap Using PriorityQueue

By default:

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
```

is Min Heap.

For Max Heap:

```
PriorityQueue<Integer> pq =
    new PriorityQueue<>(Collections.reverseOrder());
```

Example:

```java
import java.util.*;

class Main {
    public static void main(String[] args) {

        PriorityQueue<Integer> pq =
            new PriorityQueue<>(Collections.reverseOrder());

        pq.add(10);
        pq.add(50);
        pq.add(20);
        pq.add(40);
        pq.add(30);

        System.out.println(pq.peek());
    }
}
```

Output:

```
50
```

44. PriorityQueue With Custom Comparator

This is extremely important for DSA.

Suppose:

```java
class Student {
    String name;
    int marks;
}
```

We want the student with the highest marks first.

We can write:

```java
PriorityQueue<Student> pq =
    new PriorityQueue<>(
        (a, b) -> b.marks - a.marks
    );
```

Now:

```
Higher marks → higher priority
```

45. Example Custom PriorityQueue

```
import java.util.*;

class Student {

    String name;
    int marks;

    Student(String name, int marks) {
        this.name = name;
        this.marks = marks;
    }
}

class Main {

    public static void main(String[] args) {

        PriorityQueue<Student> pq =
            new PriorityQueue<>(
                (a, b) -> b.marks - a.marks
            );

        pq.add(new Student("A", 80));
        pq.add(new Student("B", 95));
        pq.add(new Student("C", 70));

        while (!pq.isEmpty()) {

            Student s = pq.poll();

            System.out.println(
                s.name + " " + s.marks
            );
        }
    }
}
```

Output:

```
B 95
A 80
C 70
```

46. Heap Sort

Heap can also be used for sorting.

For ascending order:

Build a Max Heap and repeatedly move the maximum element to the end.

Example:

```
[4, 10, 3, 5, 1]
```

Build Max Heap:

```
[10, 5, 3, 4, 1]
```

Move 10 to end:

```
[1, 5, 3, 4, 10]
```

Heapify remaining:

```
[5, 4, 3, 1, 10]
```

Continue.

Final:

```
[1, 3, 4, 5, 10]
```

47. Java Heap Sort

```java
class HeapSort {

    static void heapSort(int[] arr) {

        int n = arr.length;

        // Build Max Heap
        for (int i = n / 2 - 1; i >= 0; i--) {
            heapify(arr, n, i);
        }

        // Move maximum to the end
        for (int i = n - 1; i > 0; i--) {

            int temp = arr[0];
            arr[0] = arr[i];
            arr[i] = temp;

            heapify(arr, i, 0);
        }
    }

    static void heapify(int[] arr, int n, int i) {

        int largest = i;

        int left = 2 * i + 1;
        int right = 2 * i + 2;

        if (left < n && arr[left] > arr[largest]) {
            largest = left;
        }

        if (right < n && arr[right] > arr[largest]) {
            largest = right;
        }

        if (largest != i) {

            int temp = arr[i];
            arr[i] = arr[largest];
            arr[largest] = temp;

            heapify(arr, n, largest);
        }
    }
}
```

Complexity:

```
Time  = O(n log n)
Space = O(1)
```

48. Kth Largest Element

Heap is heavily used for:

```
Kth Largest
Kth Smallest
```

Suppose:

```
nums = [3,2,1,5,6,4]
k = 2
```

Answer:

```
5
```

One approach:

Use a Min Heap of size k.

49. Kth Largest Logic

Process:

```
3
2
1
5
6
4
```

Maintain only the largest k elements.

For:

```
k = 2
```

Heap eventually contains:

```
5
6
```

Because it's a Min Heap:

```
root = 5
```

Therefore:

```
5 = 2nd largest
```

50. Java Kth Largest

```java
class Solution {

    public int findKthLargest(int[] nums, int k) {

        PriorityQueue<Integer> pq =
            new PriorityQueue<>();

        for (int num : nums) {

            pq.add(num);

            if (pq.size() > k) {
                pq.poll();
            }
        }

        return pq.peek();
    }
}
```

Complexity:

```
Time = O(n log k)

Space = O(k)
```

This is a very important heap pattern.

51. Kth Smallest

For Kth Smallest, we can use a:

```
Max Heap
```

Keep only k smallest elements.

Example:

```
[7,10,4,3,20,15]
k = 3
```

3rd smallest:

```
7
```

Max Heap maintains:

```
3
4
7
```

Root:

```
7
```

52. Java Kth Smallest

```java
class Solution {

    public int kthSmallest(int[] nums, int k) {

        PriorityQueue<Integer> pq =
            new PriorityQueue<>(
                Collections.reverseOrder()
            );

        for (int num : nums) {

            pq.add(num);

            if (pq.size() > k) {
                pq.poll();
            }
        }

        return pq.peek();
    }
}
```

53. Top K Elements Pattern

Heap is frequently used for:

```
Top K Largest
Top K Smallest
Kth Largest
Kth Smallest
```

The basic pattern is:

Finding K largest

Use:

```
Min Heap
```

of size k.

Finding K smallest

Use:

```
Max Heap
```

of size k.

Remember this.

54. Merge K Sorted Arrays / Lists

Another major heap application.

Suppose:

```
A = [1,4,7]

B = [2,5,8]

C = [3,6,9]
```

We want:

```
[1,2,3,4,5,6,7,8,9]
```

Instead of comparing everything, put the first element of each array into a Min Heap:

```
1
2
3
```

Remove:

```
1
```

Then add its next element:

```
4
```

Heap:

```
2
3
4
```

Continue.

Complexity:

```
O(N log k)
```

where:

```
N = total elements
k = number of arrays
```

55. Find Median Using Two Heaps

This is a famous advanced heap problem.

We use:

Max Heap

For the smaller half.

Min Heap

For the larger half.

Example:

```
1 2 3 4 5
```

Smaller half:

```
1 2
```

Larger half:

```
3 4 5
```

For even numbers, the two heaps are balanced.

This allows median to be found efficiently.

56. Two Heap Structure

```
Max Heap            Min Heap

  2                   4
 / \                 / \
1                     5
```

Max Heap stores smaller half.

Min Heap stores larger half.

Median:

- If sizes equal → average of both roots
- Otherwise → root of larger heap

57. Java Median Finder

```java
class MedianFinder {

    PriorityQueue<Integer> left;
    PriorityQueue<Integer> right;

    MedianFinder() {

        left = new PriorityQueue<>(
            Collections.reverseOrder()
        );

        right = new PriorityQueue<>();
    }

    public void addNum(int num) {

        if (left.isEmpty() || num <= left.peek()) {
            left.add(num);
        } else {
            right.add(num);
        }

        if (left.size() > right.size() + 1) {
            right.add(left.poll());
        }

        if (right.size() > left.size() + 1) {
            left.add(right.poll());
        }
    }

    public double findMedian() {

        if (left.size() == right.size()) {

            return (left.peek() + right.peek()) / 2.0;
        }

        if (left.size() > right.size()) {
            return left.peek();
        }

        return right.peek();
    }
}
```

58. Priority Queue Is Not FIFO

This is important.

Normal Queue:

```
10
20
30
```

Remove:

```
10
```

PriorityQueue:

```
30 priority 1
10 priority 3
20 priority 2
```

The element with the highest priority comes out first.

Therefore:

PriorityQueue != normal Queue

59. Important PriorityQueue Warning

This:

```
System.out.println(pq);
```

does not necessarily print elements in sorted order.

For example:

```
PriorityQueue<Integer> pq = new PriorityQueue<>();

pq.add(10);
pq.add(5);
pq.add(20);
pq.add(1);
pq.add(15);

System.out.println(pq);
```

You should not assume the printed representation is:

```
[1, 5, 10, 15, 20]
```

The heap only guarantees:

```
minimum at the root
```

To process in priority order:

```
while (!pq.isEmpty()) {
    System.out.println(pq.poll());
}
```

60. Heap vs PriorityQueue

This distinction is useful.

Heap

The underlying data structure.

Example:

```
Min Heap
Max Heap
```

PriorityQueue

An abstract data structure / ADT that can be implemented using a heap.

In Java:

```
PriorityQueue<Integer>
```

uses a heap internally.

Think:

```
Priority Queue
      ↓
   Heap
      ↓
Array-based implementation
```

61. Heap vs BST

Feature	Heap	BST
Root	Min/Max	Depends
Ordering	Parent-child	Left-root-right
Search	O(n)	O(log n) average
Minimum	O(1) in Min Heap	O(log n)
Maximum	O(1) in Max Heap	O(log n)
Insert	O(log n)	O(log n) average
Delete root	O(log n)	O(log n)
Main use	Priority	Searching

62. Heap vs Stack vs Queue

Structure	Main rule
Stack	LIFO
Queue	FIFO
Priority Queue	Priority
Heap	Maintains min/max property

Example:

Stack:

```
10 → 20 → 30
pop = 30
```

Queue:

```
10 → 20 → 30
remove = 10
```

Min Priority Queue:

```
30, 10, 20
poll = 10
```

Max Priority Queue:

```
30, 10, 20
poll = 30
```

63. Important Heap Concepts You Should Know

For your DSA preparation, make sure you understand these:

Basic

1. What is a Heap? 2. Complete Binary Tree 3. Heap Property 4. Min Heap 5. Max Heap 6. Heap vs BST 7. Array representation 8. Parent/child formulas

Operations

9. Insert 10. Heapify Up 11. Peek 12. Delete Min 13. Delete Max 14. Heapify Down 15. Build Heap

Advanced

16. Bottom-up heap construction 17. Heap Sort 18. Priority Queue 19. Custom Comparator 20. Kth Largest 21. Kth Smallest 22. Top K problems 23. Merge K sorted arrays/lists 24. Two Heaps 25. Median of Data Stream

64. Most Important Java PriorityQueue Template

Min Heap

```java
PriorityQueue<Integer> pq = new PriorityQueue<>();
```

Max Heap

```java
PriorityQueue<Integer> pq =
    new PriorityQueue<>(Collections.reverseOrder());
```

Insert

```
pq.add(x);
```

Peek

```
pq.peek();
```

Remove

```
pq.poll();
```

Size

```
pq.size();
```

Empty

```
pq.isEmpty();
```

Process all

```
while (!pq.isEmpty()) {
    System.out.println(pq.poll());
}
```

65. The Most Important Heap Pattern

When you see a problem containing:

```
K largest / K smallest
```

immediately think:

HEAP

When you see:

```
Repeatedly get minimum/maximum
```

think:

PRIORITY QUEUE

When you see:

```
Merge K sorted things
```

think:

MIN HEAP

When you see:

```
Median from a stream
```

think:

TWO HEAPS

66. Final Mental Picture

You should remember Heap like this:

```
                  HEAP
                   |
            Complete Binary Tree
                   |
        +----------+----------+
        |                     |
    Min Heap              Max Heap
        |                     |
  Smallest root          Largest root
        |                     |
    Parent <=             Parent >=
     children              children
        |                     |
        +----------+----------+
                   |
                 Array
                   |
     +-------------+-------------+
     |             |             |
  Insert        Delete        Peek
     |             |             |
 O(log n)      O(log n)      O(1)
```

And:

```
               PRIORITY QUEUE
                    |
                  Heap
                    |
        +----------+----------+
        |                     |
  Min Priority          Max Priority
     Queue                 Queue
        |                     |
  smallest               largest
  first                  first
```

The formulas you absolutely must memorize

For a 0-indexed heap:

```
Parent      = (i - 1) / 2
Left child  = 2 * i + 1
Right child = 2 * i + 2
```

The core operations

```
Insert       → Heapify Up
Delete Root  → Heapify Down
Build Heap   → Bottom-Up Heapify
```

Complexity

```
Peek         → O(1)
Insert       → O(log n)
Delete       → O(log n)
Build Heap   → O(n)
Heap Sort    → O(n log n)
Search       → O(n)
```